

SOFTWARE REQUIREMENTS SPECIFICATION

CampusOS

A Verified, Reputation-Driven Campus Super-App

Document Version: 1.0

Prepared for: Single-Campus Pilot Deployment

Date: July 2026

Classification: Founder / Engineering Working Document

Table of Contents

1. Introduction
2. Overall Description
3. System Features (Functional Requirements)
4. Data Requirements (Core Schema Overview)
5. External Interface Requirements
6. Non-Functional Requirements
7. System Guardrails & Risk Mitigation ("Reality Check")
8. Appendices

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for CampusOS, a centralized, verification-first digital ecosystem intended to replace fragmented student communication and utility tools (WhatsApp groups, physical noticeboards, scattered marketplace pages, ad-hoc spreadsheets) with a single trusted platform. It is intended to guide design, development, testing, and stakeholder sign-off across all build phases.

1.2 Document Conventions

- Requirement IDs follow the pattern MOD-FR-### (e.g., AUTH-FR-001), grouped by module.
- Priority levels used throughout: Must-Have (P0), Should-Have (P1), Nice-to-Have (P2).
- "Student" and "User" are used interchangeably unless a specific role is named.
- All Time-Credit values are virtual, non-cash units unless explicitly stated otherwise.

1.3 Intended Audience

This document is written for the founding/developer team (technical design and sprint planning), a faculty or Student Council sponsor (approval authority for moderation-related features), evaluators or hackathon judges (scope and rigor assessment), and future contributors onboarding onto the codebase.

1.4 Project Scope

CampusOS is a web-first, mobile-responsive Progressive Web Application scoped initially to a single college domain. In scope: domain-locked authentication, a reputation and permission engine, an Academic Ledger (calendar + resource sharing + department communities), a Utility Marketplace (bazaar + lost & found), a peer-to-peer Skill Barter economy with escrow-style credits, a Recruitment Portal, and a universal Sentiment/Rating Engine. Out of scope for v1: real-money payment processing (Marketplace deals are peer-arranged off-platform via cash/UPI), native mobile apps (a responsive PWA substitutes for this), and multi-campus federation (the architecture allows it later, but v1 targets one .edu.in domain).

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
RLS	Row Level Security — a PostgreSQL/Supabase feature enforcing per-row access rules at the database layer.
Reputation Score	A numeric trust score computed from a user's verified positive actions (ratings received, completed trades, approved contributions).
Time-Credit	A non-monetary unit earned by teaching/helping and spent to request help; underpins the Skill Barter module.
Escrow-style Hold	A logical (non-custodial) hold placed on a Time-Credit balance until both parties confirm session completion.
Interactions Engine	The shared, polymorphic backend service handling all ratings, reviews, and comments across every module.
Reader Tier	Default, lowest-trust permission tier assigned to every newly verified account.

Term	Definition
PYQ	Previous Year Question paper — a common academic resource type shared in the Academic Ledger.
PWA	Progressive Web App — an installable, offline-capable web application.

1.6 References

- Supabase official documentation (Auth, Realtime, Storage, RLS policies).
- Next.js 15 App Router documentation.
- IEEE Std 830-1998 — structural reference for this document.
- India's Digital Personal Data Protection (DPDP) Act, 2023 — referenced in Section 6.7.

2. Overall Description

2.1 Product Perspective

CampusOS is a new, self-contained product rather than an extension of an existing system. It replaces a fragmented toolchain (departmental WhatsApp groups, Google Forms lost-and-found threads, OLX/Facebook Marketplace for second-hand goods, and word-of-mouth for project teams) with one identity-verified system where trust is a first-class, computable property of every account. The system is built on a Backend-as-a-Service model (Supabase/PostgreSQL) fronted by a Next.js application, which keeps infrastructure overhead low enough for a student-run team to operate and iterate quickly.

2.2 Product Functions — Summary

Module	Primary Function	Stickiness Mechanism
Auth & Identity	College-domain-locked signup and login	Removes anonymity; is the trust foundation for every other module
Reputation Engine	Computes trust score; gates permissions	Turns good behaviour into unlocked capability
Academic Ledger	Calendar, resource sharing, dept. communities	Contextual, department/year-filtered feed
Marketplace	Second-hand bazaar + Lost & Found	Instant-match alerts on wanted items
Skill Barter	Time-Credit peer tutoring/help exchange	Escrow-backed fairness; credits feel earned
Recruitment Portal	Skill-tag based teammate matching	Proactive notification of relevant openings
Sentiment Engine	Universal rating/review layer	Surfaces "Top Rated" classes, labs, events

2.3 User Classes and Characteristics

User Class	Description	Technical Proficiency
Reader (default)	Newly verified student; can browse, comment, and rate	Low-Medium
Contributor	Reader who has crossed the reputation threshold to post resources/listings	Medium
Trader	Contributor with sufficient reputation and zero unresolved reports, permitted to list on Marketplace and offer Skill Barter sessions	Medium
Department/Council Moderator	Elected or appointed student who approves calendar submissions and department content	Medium
Admin (Faculty/Core Team)	Full platform oversight: user management, report resolution, global announcements	Medium-High

2.4 Operating Environment

- Client: any evergreen browser (Chrome, Edge, Safari, Firefox) on desktop or mobile; installable as a PWA on Android and iOS.

- Server: Supabase-managed PostgreSQL, Auth, Storage, and Realtime services (cloud-hosted, region selected close to campus for latency).
- Hosting/CDN: Netlify (primary, for rapid iteration and preview deployments) or Azure Static Web Apps (if institutional/Azure-for-Students credits are available).
- AI services: Google Gemini API for the optional coaching/matching-copy features (rate-limited, server-side only).

2.5 Design and Implementation Constraints

- Authentication must reject any email not matching the verified institutional domain pattern (e.g., @yourcollege.edu.in).
- All data access must be mediated by Postgres RLS policies — no table may rely solely on client-side checks.
- The team is small/student-run: the stack must minimize DevOps overhead (managed services over self-hosted infrastructure).
- Must operate acceptably on campus Wi-Fi with variable bandwidth; heavy assets (images) must be compressed and lazy-loaded.

2.6 Assumptions and Dependencies

- The institution issues every student a unique, non-transferable @college domain email for the lifetime of their enrolment.
- A Student Council or faculty sponsor is available to act as the moderation/approval authority for the Academic Calendar.
- Supabase's free/pro tier limits (connections, storage, Realtime concurrent connections) are sufficient for a single-campus pilot (validated in Section 6.4).

3. System Features (Functional Requirements)

This section enumerates functional requirements per module. Each requirement has a unique ID, priority, description, triggers/inputs, expected system response, and governing business rules.

3.1 Module: Authentication & Identity Verification

This module is the trust foundation of the entire platform: every other module assumes the acting user has passed domain-locked verification.

AUTH-FR-001 Domain-Locked Signup [Priority: P0]

The system shall only accept registrations where the email domain exactly matches the whitelisted institutional domain (e.g., @yourcollege.edu.in).

- Inputs / Triggers: Full name, institutional email, password (or OAuth via institutional Google Workspace if available), department, year of study.
- Outputs / System Response: A verification email with a one-time confirmation link/OTP; account remains in 'unverified' state until confirmed.

Business Rules:

- Signup requests from any non-whitelisted domain are rejected at the API layer before an Auth record is created.
- Free/personal email providers (gmail.com, yahoo.com, etc.) are always rejected regardless of local-part.
- A single institutional email may only be associated with one active account.

AUTH-FR-002 Email/OTP Verification [Priority: P0]

The system shall require confirmation of email ownership before any profile record is activated.

- Inputs / Triggers: Click of confirmation link or entry of a 6-digit OTP within 15 minutes of issuance.
- Outputs / System Response: Account status transitions from 'unverified' to 'active'; a public.profiles row is auto-created via a database trigger on auth.users insert.

Business Rules:

- OTPs expire after 15 minutes and are single-use.
- Unverified accounts are purged after 7 days of inactivity.

AUTH-FR-003 Session Management & Login [Priority: P0]

The system shall issue and refresh JWT-based sessions via Supabase Auth, with secure, http-only cookie storage on the Next.js server.

- Inputs / Triggers: Email + password, or magic-link/OTP login.
- Outputs / System Response: Signed JWT (access + refresh token pair) scoped to the user's id and role claims.

Business Rules:

- Access tokens expire in 1 hour; refresh tokens rotate on use.
- Five consecutive failed logins trigger a temporary 15-minute logout.

AUTH-FR-004 Profile Auto-Sync Trigger [Priority: P0]

A Postgres trigger shall automatically create a corresponding row in public.profiles whenever a new row is inserted into auth.users, pre-populating department, year, default reputation_score (0), and default permission tier ('Reader').

Business Rules:

- This trigger is the single source of truth for profile creation — no client-side insert path into public.profiles is permitted.

AUTH-FR-005 Password Reset & Account Recovery [Priority: P1]

The system shall allow users to reset a forgotten password via a verified-email-only reset link.

Business Rules:

- Reset links expire after 30 minutes and invalidate all existing refresh tokens on use.

3.2 Module: Reputation & Permission Engine

The reputation engine is a cross-cutting service consumed by every other module; it converts verified positive actions into a numeric score and maps score bands to permission tiers.

REP-FR-001 Reputation Score Computation [Priority: P0]

The system shall maintain a single reputation_score per profile, recalculated (or incrementally updated) whenever a scoring event occurs.

- Inputs / Triggers: Scoring events: resource upvote received, average rating received on a listing/session, event attendance confirmation, completed barter session, resolved report against the user (negative), content removed by moderator (negative).
- Outputs / System Response: Updated reputation_score persisted on public.profiles; a reputation_events audit row is inserted for every change.

Business Rules:

- Score changes are always logged with a reason code — no silent mutation of reputation_score is permitted.
- Negative events (upheld reports, content takedowns) subtract points and can demote a user's tier.

REP-FR-002 Permission Tier Gating [Priority: P0]

The system shall gate access to write-heavy or risk-bearing actions (posting a Marketplace listing, offering a Skill Barter session, submitting a Calendar event for approval) behind minimum reputation thresholds.

- Outputs / System Response: Attempted actions below the required tier are blocked at both the UI (disabled state with explanation) and the RLS policy layer (hard reject).

Business Rules:

- Tier thresholds: Reader (0+, default) → Contributor (25+ points and ≥3 approved comments/ratings) → Trader (75+ points, 0 open reports) → Council/Admin (manually assigned, not score-derived).
- Tier and threshold values are stored in a config table, not hard-coded, so they can be tuned without a deploy.

REP-FR-003 Public Reputation Display [Priority: P1]

The system shall display a user's tier badge and aggregate rating on their profile and next to their listings/comments, without exposing the raw internal scoring formula.

3.3 Module: Academic Ledger & Community

3.3.1 Unified Calendar

CAL-FR-001 Event Submission [Priority: P0]

Any Contributor-tier or above user shall be able to submit an event/deadline with title, description, date/time, department tags, and an optional banner image.

- Outputs / System Response: Event is created in a 'pending' state, invisible to the global feed until approved.

CAL-FR-002 Council/Admin Approval Workflow [Priority: P0]

Department Moderators or Admins shall review pending events and approve, reject (with reason), or request edits.

Business Rules:

- Approved events become globally visible and are pushed to relevant department feeds.
- Rejected events notify the submitter with the stated reason; resubmission is allowed.

CAL-FR-003 Personalized Calendar View [Priority: P1]

The system shall render a filtered calendar view prioritizing the viewer's own department and year, with an option to browse all-campus events.

3.3.2 Department-Specific Communities

COMM-FR-001 Auto-Enrolment into Department Channel [Priority: P0]

On profile creation, a user shall be automatically subscribed to their department's real-time chat channel (via Supabase Realtime) based on the department field captured at signup.

COMM-FR-002 Channel Messaging [Priority: P0]

Users shall be able to post text messages, links, and small file attachments (resource snippets) within their department channel(s), with messages persisted and paginated.

Business Rules:

- Only members of a department (or Admins) can read/write that department's channel — enforced via RLS on channel_members.

3.3.3 Resource Sharing (Notes / PYQs / Projects)

RES-FR-001 Upload Academic Resource [Priority: P0]

Contributor-tier users shall upload notes, previous-year question papers, or project references tagged by subject, department, and year.

- Outputs / System Response: File stored in a private Supabase Storage bucket; access mediated by signed URLs generated on request.

RES-FR-002 Resource Rating & Reputation Feed-in [Priority: P1]

Every downloaded resource shall be rateable via the shared Interactions Engine (Section 3.7); high cumulative ratings feed positively into the uploader's reputation_score.

3.3.4 Event / Lab Rating System

SENT-FR-001 Rate a Class, Lab, or Event [Priority: P1]

Students shall submit a 1–5 star rating plus optional text feedback against any completed class, lab session, or event, using the shared Interactions Engine.

Business Rules:

- Ratings are anonymous to other students but attributable internally for abuse investigation.
- A user may rate a given entity only once per offering/instance.

SENT-FR-002 "Top Rated" Surfacing [Priority: P2]

The system shall compute and display a rolling "Top Rated" list of classes/labs/events per department, refreshed on a scheduled job (e.g., nightly materialized view refresh).

3.4 Module: The Utility Marketplace

3.4.1 Second-Hand Bazaar

MKT-FR-001 Create Listing [Priority: P0]

Trader-tier users shall create a listing with title, category (books/electronics/misc), price, condition, description, and up to 5 images.

Business Rules:

- Listing creation is blocked for users below Trader tier or with an open unresolved report.

MKT-FR-002 Search & Filter Listings [Priority: P0]

Users shall search/filter listings by category, department relevance, price range, and keyword, with results paginated and sorted by recency or relevance.

MKT-FR-003 Instant-Match Alerts ("Wanted") [Priority: P1]

A user shall be able to register a 'wanted' request (e.g., a specific textbook ISBN/title); when a matching listing is created, the system shall push a real-time/webhook notification to that user.

- Outputs / System Response: Notification delivered via in-app toast + (optionally) email/push.

MKT-FR-004 Mark Listing as Sold/Closed [Priority: P0]

The listing owner shall mark a listing as Sold, removing it from active search results while retaining it for the buyer's/seller's transaction history.

3.4.2 Lost & Found

LF-FR-001 Submit Lost or Found Report [Priority: P0]

Any verified user (Reader tier and above) shall submit a Lost or Found item report with description, location, timestamp, and optional photo.

Business Rules:

- Lost & Found is intentionally open to Reader tier (lower barrier) since it is low-risk and high community value.

LF-FR-002 Status Tracking [Priority: P0]

Each Lost & Found record shall carry a status of Active or Resolved/Found, with status changes logged and timestamped.

3.4.3 Safety Features

SAFE-FR-001 Report Listing / User [Priority: P0]

Any user shall be able to flag a listing, message, or profile for review, selecting a reason (scam, inappropriate, spam, harassment) and optional free-text detail.

- Outputs / System Response: A report row is created with status 'open' and routed to the Admin moderation queue.

Business Rules:

- Three or more concurrent open reports against the same listing shall auto-hide it pending review.
- Repeated upheld reports against a user reduce reputation_score and can suspend Trader privileges.

SAFE-FR-002 Peer-to-Peer Transaction Log [Priority: P1]

When a Marketplace deal is marked complete by both parties, the system shall log a transaction record (listing, buyer, seller, timestamp) used later for dispute resolution and reputation calculation.

3.5 Module: Peer-to-Peer Skill Barter (Time-Credit Economy)

This module implements a closed-loop, non-monetary economy: teaching earns Time-Credits, which can be redeemed for help elsewhere on the platform. An escrow-style mechanism protects both parties from a session being abandoned mid-way.

BAR-FR-001 Create a Skill Offer [Priority: P1]

Trader-tier users shall list a skill they can teach/help with, tagged (e.g., "Next.js", "Linear Algebra"), with an indicative time commitment.

BAR-FR-002 Request a Skill Session [Priority: P1]

A requester shall book a session against an offer, provided their Time-Credit balance (or a zero-balance first-session allowance) permits it.

Business Rules:

- New accounts receive a one-time starter credit (e.g., 1 credit) to bootstrap participation without gaming the system.

BAR-FR-003 Escrow-Style Hold on Booking [Priority: P0]

Upon session confirmation, the system shall place a logical hold equal to the agreed credit amount on the requester's balance (deducted from spendable balance, not yet transferred to the teacher).

- Outputs / System Response: A row in `escrow_transactions` with status 'held'.

Business Rules:

- Held credits are not spendable elsewhere until the hold is released (completed) or reversed (cancelled/disputed).

BAR-FR-004 Dual Confirmation of Completion [Priority: P0]

Both the teacher and the requester shall independently confirm session completion within 48 hours of the scheduled end time.

Business Rules:

- When both confirm, the hold is released: credit transfers from requester to teacher, and both parties are prompted to rate each other via the Interactions Engine.
- If only one party confirms after 48 hours, the session is flagged for lightweight Admin review rather than auto-resolved, to deter one-sided abuse.

BAR-FR-005 Dispute / Cancellation Handling [Priority: P1]

Either party shall raise a dispute before dual confirmation, freezing the held credits pending Admin review.

Business Rules:

- Cancellations made more than 2 hours before the scheduled session incur no penalty; late cancellations may cost the cancelling party a small reputation deduction.

BAR-FR-006 Credit Ledger & Balance View [Priority: P1]

Users shall view a full, append-only ledger of credits earned, spent, held, and released, plus their current spendable balance.

3.6 Module: Recruitment Portal

REC-FR-001 Post a Team/Project Opening [Priority: P1]

Contributor-tier users shall post an opening (hackathon team, project, fest committee) with required skill tags, headcount, and deadline.

REC-FR-002 Skill-Tag Matching & Proactive Notification [Priority: P1]

The system shall match new openings against each user's declared skill tags (from their profile) and proactively notify qualified users of relevant new postings.

- Outputs / System Response: In-app notification and/or webhook-triggered email digest.

Business Rules:

- Matching is tag-intersection based in v1 (exact/synonym match); a weighted relevance model is a Phase 3+/roadmap enhancement.

REC-FR-003 Apply to an Opening [Priority: P1]

A user shall apply to an opening with an optional short note; the poster sees a ranked list of applicants (ranked by profile reputation_score and tag overlap).

REC-FR-004 Accept/Reject Applicant [Priority: P1]

The poster shall accept or reject applicants; acceptance auto-creates a private chat thread (reusing the Realtime channel infrastructure from Section 3.3.2).

3.7 Module: Universal Interactions Engine (Sentiment Engine)

Rather than building separate review systems per module, CampusOS implements a single polymorphic interactions engine that every module (Resources, Events/Labs, Marketplace listings, Skill Barter sessions, user profiles) writes to and reads from. This keeps the schema normalized and lets reputation scoring, moderation, and analytics operate over one consistent data model.

INT-FR-001 Polymorphic Review Submission [Priority: P0]

The system shall accept a review/rating referencing any entity_type (resource, event, listing, barter_session, profile) and entity_id, with a numeric rating and optional text.

Business Rules:

- One review per (author, entity_type, entity_id) — enforced by a unique constraint to prevent review-stuffing.

INT-FR-002 Aggregate Rating Computation [Priority: P0]

The system shall maintain (or compute on read via a view) average rating and count per entity, used for display and for reputation feed-in.

INT-FR-003 Comment Threads [Priority: P1]

The engine shall support threaded (single-level reply) comments on the same entities, independent of the numeric rating.

INT-FR-004 Moderation Hooks [Priority: P1]

Any review/comment shall be reportable via the same Report mechanism defined in SAFE-FR-001, and removable by an Admin, with removal logged for audit.

3.8 Module: Notification System

NOTIF-FR-001 Event-Driven Notifications [Priority: P1]

The system shall generate notifications for: Skill Match (REC-FR-002), Marketplace Instant-Match (MKT-FR-003), Calendar approval/rejection, Barter session reminders and confirmations, and Report status updates.

- Outputs / System Response: In-app notification center (read/unread state) plus optional email via a transactional email provider triggered by a Supabase Database Webhook.

NOTIF-FR-002 Notification Preferences [Priority: P2]

Users shall toggle categories of notification on/off from their profile settings.

4. Data Requirements (Core Schema Overview)

CampusOS uses a highly normalized PostgreSQL schema (via Supabase). Below is the core entity set; each table is protected by Row Level Security policies scoped to `public.profiles.id` (mirroring `auth.uid()`).

Table	Key Columns	Purpose
profiles	id (PK, = <code>auth.users.id</code>), department, year, reputation_score, permission_tier, skill_tags[]	Central user record; auto-synced from <code>auth.users</code> via trigger.
reputation_events	id, profile_id (FK), delta, reason_code, created_at	Append-only audit log of every reputation change.
calendar_events	id, submitted_by (FK), title, dept_tags[], status(pending/approved/rejected), start_at	Academic calendar submissions and approvals.
channels / channel_messages	channel_id, department, sender_id (FK), body, created_at	Real-time department community chat.
resources	id, uploader_id (FK), subject, dept, year, file_path, storage_bucket	Notes / PYQs / project files (Storage-backed).
listings	id, seller_id (FK), category, price, condition, status(active/sold), images[]	Second-hand Bazaar items.
lost_found_items	id, reporter_id (FK), type(lost/found), status(active/found), location, occurred_at	Lost & Found tracker.
reports	id, reporter_id (FK), entity_type, entity_id, reason, status(open/resolved)	Universal safety/report queue.
transactions	id, listing_id (FK), buyer_id (FK), seller_id (FK), completed_at	Peer-to-peer Marketplace transaction log.
credit_ledger	id, profile_id (FK), type(earn/spend/hold/release), amount, ref_session_id	Append-only Time-Credit ledger.
barter_sessions	id, teacher_id (FK), requester_id (FK), skill_tag, status, scheduled_at	Skill Barter bookings and lifecycle state.
escrow_transactions	id, session_id (FK), amount, status(held/released/reversed)	Escrow-style hold state for barter credits.
openings	id, posted_by (FK), title, skill_tags[], headcount, deadline	Recruitment Portal postings.
applications	id, opening_id (FK), applicant_id (FK), status(pending/accepted/rejected)	Applications against Recruitment Portal openings.
interactions	id, author_id (FK), entity_type, entity_id, rating, comment, parent_id	Universal polymorphic ratings/reviews/comments engine.
notifications	id, profile_id (FK), type, payload(jsonb), read_at	In-app notification feed.

4.1 Indexing & Referential Integrity Notes

- Every FK column referencing `profiles.id` is indexed to keep per-user dashboard queries ("my listings", "my ledger", "my applications") fast as data grows.

- All *_at timestamp columns used in feeds/sorting (created_at, scheduled_at, occurred_at) are indexed, per the scale guardrail in Section 7.3.
- interactions.entity_type + interactions.entity_id form a composite index to make "all reviews for entity X" lookups $O(\log n)$.
- A unique constraint on (author_id, entity_type, entity_id) in interactions prevents duplicate reviews (see INT-FR-001).

5. External Interface Requirements

5.1 User Interfaces

- Responsive design (mobile-first) built with Tailwind CSS; a fixed bottom navigation bar on mobile mirrors the six core modules.
- Dashboard/home feed personalizes content by department and year, per COMM-FR-001 and CAL-FR-003.
- All destructive or trust-affecting actions (reporting, cancelling a barter session, deleting a listing) require an explicit confirmation step.

5.2 Hardware Interfaces

No dedicated hardware is required for v1. The optional "Live Campus Heatmap" mentioned in the long-term roadmap (Section 8.3) would require IoT occupancy sensors or Wi-Fi AP density data — explicitly out of scope until a later phase.

5.3 Software Interfaces

Interface	Technology	Purpose
Auth / DB / Storage / Realtime	Supabase (PostgreSQL)	Core backend-as-a-service: authentication, relational data, file storage, live subscriptions.
AI Coaching / Copy Assist	Google Gemini API	Optional server-side calls for coaching prompts and smart-match copy summaries.
Transactional Email	Resend / Postmark (or Supabase's SMTP integration)	OTP emails, approval/rejection notices, digest notifications.
Hosting / CI	Netlify (primary) or Azure Static Web Apps	Build, preview deployments, and production hosting.
Webhooks	Supabase Database Webhooks → Next.js API routes	Triggers Notification System events (Section 3.8) on row changes.

5.4 Communication Interfaces

- HTTPS/TLS 1.2+ for all client-server traffic; no plaintext endpoints permitted.
- WebSocket-based Realtime channels (Supabase Realtime) for department chat and live notification delivery.
- Signed, time-limited URLs for all private Storage bucket access (resources, listing images, ID-adjacent documents if ever required).

6. Non-Functional Requirements

6.1 Performance

- P95 page load (server-rendered dashboard) under 2.5s on a typical campus 4G/Wi-Fi connection.
- P95 API response time under 400ms for indexed, single-row-scope queries (per-user dashboards, listing fetch).
- Realtime message delivery latency under 1s for department chat under normal load.

6.2 Security

- Every table exposed to the client has an explicit RLS policy; default-deny is the starting posture for every new table.
- JWTs are validated server-side on every API Route Handler; no route trusts a client-supplied user id without cross-checking the session.
- Rate limiting on signup, login, and report-submission endpoints to blunt automated abuse.
- All file uploads are virus/type-scanned or at minimum extension- and MIME-type restricted before storage.

6.3 Reliability & Availability

- Target 99.5% uptime for the pilot phase, consistent with managed-service SLAs from Supabase and Netlify/Azure.
- Nightly automated database backups (Supabase point-in-time recovery where available on plan).

6.4 Scalability

- Schema and indexing (Section 4.1) are designed to remain performant from a pilot cohort (hundreds of users) up to full-campus scale (tens of thousands).
- Stateless Next.js API routes allow horizontal scaling behind the hosting provider's edge/CDN layer without code changes.

6.5 Usability & Accessibility

- Core flows (signup, post a listing, rate an event) must be completable in under 5 taps/clicks from the home screen.
- Colour contrast and font sizing target WCAG 2.1 AA where practical for a student-built v1.

6.6 Maintainability

- Feature modules are organized as separate route groups in the Next.js App Router to allow independent iteration without cross-module regressions.
- Config-driven reputation thresholds and permission tiers (REP-FR-002) avoid hard-coded business rules scattered across the codebase.

6.7 Legal / Compliance

- Personal data handling aligns with India's Digital Personal Data Protection (DPDP) Act, 2023: a clear consent notice at signup, a stated data-retention policy, and a user-accessible data-deletion request path.
- Institutional sponsorship (Section 2.6) is recommended to formalize the platform's standing relative to student code-of-conduct policies, especially for the Marketplace and reporting features.

7. System Guardrails & Risk Mitigation ("Reality Check")

7.1 The Cold-Start Problem

A campus network platform is worthless with too few users. CampusOS mitigates this by sequencing value delivery: the Marketplace and Lost & Found are opened at Reader tier (lowest barrier) because they deliver immediate, individually-felt value ("I found a cheap textbook") even with a small initial user base, without requiring network effects across every module simultaneously.

7.2 Spam & Bad-Actor Control

New accounts default to Reader tier and cannot post listings, offer barter sessions, or submit calendar events directly. They must first participate in low-risk, reversible actions — rating, commenting — which both builds a visible track record and naturally filters out throwaway or bad-faith accounts before they reach trade-capable tiers (REP-FR-002).

7.3 Scale Guardrails

All high-traffic tables index on user-scoped foreign keys and timestamp columns (Section 4.1) so that feed and dashboard queries stay performant as the user base and content volume grow from a single-department pilot to full-campus adoption.

7.4 Abuse & Dispute Handling

- The Report mechanism (SAFE-FR-001) and Escrow dispute path (BAR-FR-005) both route to a single Admin moderation queue rather than being handled ad hoc per module.
- Automatic soft-actions (auto-hide at 3+ concurrent open reports) reduce the window of harm before a human moderator acts.

8. Appendices

8.1 Reputation Score — Illustrative Formula

$\text{reputation_score} = \Sigma(\text{rating_events} \times \text{weight}) + (\text{approved_contributions} \times w1) + (\text{completed_barter_sessions} \times w2) - (\text{upheld_reports} \times \text{penalty})$. Exact weights ($w1$, $w2$, penalty) are stored in a tunable config table, not hard-coded, and should be calibrated during the Phase 3 pilot based on observed behaviour.

8.2 Permission Tier Reference

Tier	Approx. Threshold	Unlocked Capabilities
Reader	0+ (default on verification)	Browse, comment, rate, submit Lost & Found.
Contributor	25+ points, 3+ approved contributions	Upload resources, submit calendar events, post Recruitment openings.
Trader	75+ points, 0 open reports	List on Marketplace, offer Skill Barter sessions.
Council/Admin	Manually assigned	Approve calendar events, resolve reports, manage users.

8.3 Long-Term Roadmap Notes (Beyond v1 Scope)

- Live campus "activity heatmap" (optional IoT/Wi-Fi density integration) referenced in the original product vision — deferred until a later phase given hardware dependency.
- Native mobile wrapper (Capacitor/Expo) once the PWA's usage data justifies the investment.
- Multi-campus federation — would require namespacing the schema by `institution_id`, a change best planned for after the single-campus pilot validates the model.

8.4 Open Questions for Stakeholder Sign-Off

- Who holds the Admin role in practice — a faculty sponsor, a Student Council body, or a rotating student moderator team?
- What is the institution's official position on peer-to-peer Marketplace transactions occurring "on campus" (liability, code-of-conduct alignment)?
- Should Time-Credits ever expire, or remain valid indefinitely across a student's enrolment?